

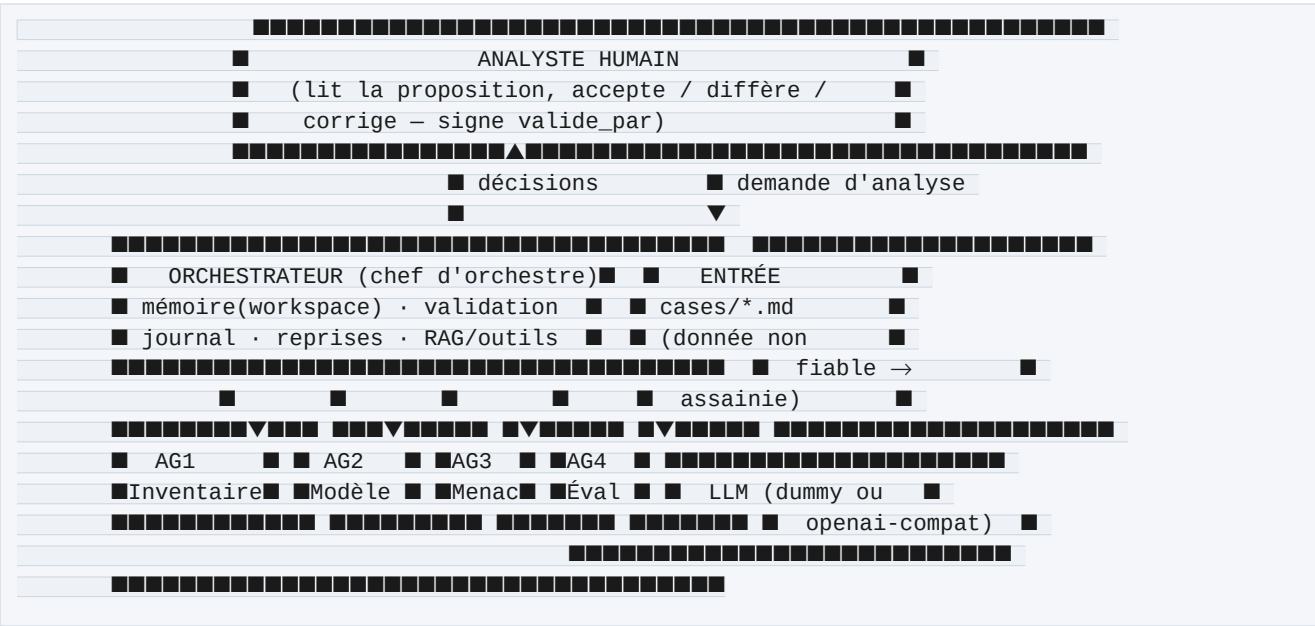
Fonctionnement interne — agents-risques (projet E21)

Document **technique** : comment l'outil fonctionne de l'intérieur — architecture, flux de données, contrats JSON, mécanismes de sécurité, et correspondance avec les garde-fous du sujet et les recommandations OWASP LLM. Pour *savoir quoi faire*, voir `GUIDE_UTILISATION.md`.

Table des matières

- 1. Vue d'ensemble
- 2. Correspondance avec la démarche du cours
- 3. Architecture des composants
- 4. Contrat d'échange : l'enveloppe JSON
- 5. Séquence détaillée d'une analyse
- 6. Garde-fous de sécurité (G1→G10)
- 7. La matrice déterministe (G7)
- 8. Reprises et arrêt bavard (G8)
- 9. Recherche de connaissances (RAG) et base de connaissances (G10)
- 10. Le snapshot CVE
- 11. Comparaison avec la référence manuelle (`--comparer`)
- 12. Journalisation (G5)
- 13. Simulateur déterministe vs fournisseur réel
- 14. Modèle de données
- 15. Validation par agent : tableau de référence
- 16. Limites et points d'attention
- 17. Glossaire

1. Vue d'ensemble



Les **5 agents ne se parlent jamais directement** : tout passe par l'orchestrateur, qui valide, normalise et mémorise chaque étape avant de fournir la suivante. Les **outils** sont distincts des agents : un registre central + une **liste blanche par agent** (`_OUTILS_PAR_AGENT`) — ce sont l'orchestrateur, jamais les agents eux-mêmes, qui invoque les outils au nom de chaque agent.

Le cours (ch. 05 et p. 109) propose une chaîne : *décrire* → *identifier* (STRIDE/ LINDDUN) → *détailler* (attaques, ATT&CK) → *prioriser* (DREAD/CVSS) → *traiter*. Le système la reproduit en 6 étapes :

3. Architecture des composants

Fichier	Rôle
config.py	constantes : échelles, matrice , traitements autorisés/interdits, tentatives max, seuils
models.py	dataclasses Actif, Menace, Evaluation, Risque (avec valide_par: str \ None)
schemas.py	les schémas JSON des échanges (enveloppe + sortie par agent)
validation.py	validations : schéma + règles métier (sources, valide_par , traitements, échelles, matrice, unicité)
sanitizer.py	filtrage anti-injection de prompt : détection, neutralisation, balisage « donnée »

Fichier	Rôle
<code>workspace.py</code>	mémoire partagée de la session (produits + statuts + registre) et persistance <code>workspace.json</code>
<code>journal.py</code>	journalisation horodatée append-only (<code>journal.jsonl</code>)

3.2 `src/tools/` — les outils (lecture seule, invoqués par l'orchestrateur)

Les agents **n'appellent aucun outil eux-mêmes** : l'orchestrateur délègue selon l'agent (`_OUTILS_PAR_AGENT` , cf. § 5) dans une `SandboxUtils` (liste blanche, `PermissionError` sinon).

Outil	Fonction	Gas
<code>lire_fichier</code>	lit un descriptif de système uniquement dans <code>cases/</code> (taille max, pas de <code>...</code> , pas d'écriture)	G2
<code>chercher_connaissance</code>	recherche RAG dans <code>knowledge/*.md</code> et renvoie extraits + références	G4, G10
<code>calculer_niveau</code>	applique la matrice $P \times I \rightarrow$ <code>niveau</code> (déterministe)	G7
<code>rechercher_cve</code>	interroge le snapshot local CVE/CVSS	G4

3.3 `src/llm/` — la couche modèle (remplaçable, G9)

Fichier	Rôle
<code>interface.py</code>	contrat <code>FournisseurLLM</code> (Protocol) + <code>extraire_json</code>
<code>dummy.py</code>	simulateur déterministe : reproduit la référence pour tester la chaîne — <code>nom_produit = "dummy-deterministic (test de chaîne - n'est PAS une analyse)"</code> ; paramètre <code>fidele=True/False</code> (False = registre valide mais divergent, cf. § 11)
<code>openai_compat.py</code>	fournisseur réel : POST <code>{base_url}/chat/completions</code> (urllib standard, sans dépendance), <code>temperature=0.2</code> par défaut

3.4 `src/agents/` — l'orchestration et les consignes

Fichier	Rôle
<code>base.py</code>	classe <code>Agent</code> : assemblage du prompt, réinjection des erreurs en reprise (<code>=== ERREURS DE LA TENTATIVE PRÉCÉDENTE ===</code>), mémorisation du dernier prompt (<code>_dernier_prompt</code>), point d'appel unique
<code>prompts.py</code>	les 5 consignes (AG1→AG5), le bloc système commun, listes d'outils et requêtes RAG
<code>agents.py</code>	les 5 classes concrètes + ordre d'exécution

Fichier	Rôle
orchestrator.py	pipeline (AG1→AG5), validation, reprises , normalisation déterministe, human loop, délégation des outils selon l'agent (<code>_OUTILS_PAR_AGENT</code>), <code>octets_entree</code> réels journalisés

3.5 `src/cli/` — le point d'entrée

Fichier	Rôle
main.py	argparse : sous-commandes <code>run</code> et <code>test-injection</code> , option <code>--comparer</code> (jalón 4), session horodatée, écriture des 3 livrables

4. Contrat d'échange : l'enveloppe JSON

Toutes les sorties d'agents obéissent à **la même enveloppe** (`core/schemas.py`) :

```
{
  "agent": "AG3",
  "resume": "...",
  "sortie": { "..."},
  "sources": ["STRIDE", "01_Cadrage (frontières)"],
  "incertitudes": []
}
```

- `sources` : **obligatoire, non vide** (G4 — parade anti-hallucination).
- `incertitudes` : facultatif — l'agent y signale ce qu'il ne sait pas.
- `valide_par` : **n'apparaît pas** dans cet échange, et quand il apparaît dans un risque il doit être `null` (G6). Le schéma AG5 impose même `"type": "null"`.

Les schémas de `sortie` par agent (cf. `SCHEMAS_SORTIE`) :

Agent	<code>sortie</code> contient	Contraintes notables
AG1	<code>actifs[]</code>	<code>id = A-\d{2}</code> , <code>type</code> parmi 6 valeurs, <code>criticite</code> ∈ {C,I,A}, <code>valeur</code> parmi 4
AG2	<code>modele_retenu</code> , <code>modeles_complementaires</code>	<code>echelles_normalisees</code> (aucun doubleton)
AG3	<code>menaces[]</code>	<code>id_menace = M-\d{2}</code> , <code>frontiere = "1"..."6"</code> , <code>categorie = STRIDE-... ou LINDDUN-...</code>
AG4	<code>evaluations[]</code>	<code>id_menace = M-\d{2}</code> , P/I dans les échelles de configuration
AG5	<code>risques[]</code>	<code>id = R-\d{2}</code> , <code>menace_id requis</code> , <code>traitement</code> ∈ 4 réponses, <code>valide_par</code> <code>null</code>

`additionalProperties: true` permet aux agents d'enrichir (ex. `menace_id`, `contraintes`), sans jamais supprimer les champs requis.

5. Séquence détaillée d'une analyse

Entrée (CLI `commande_run`) : 1. `lire_fichier("cases/casB_mediconsult.md")` — lecture seule, chemin restreint, $\leq 100\,000$ octets.

Phase 1 — assainissement (`Orchestrateur.analyser`) : 2. `sanctionner_document(description)` : a. **normalisation préalable** du texte (homoglyphes cyrilliques/grecs/fullwidth → latin, caractères zero-width et diacritiques combinants retirés, espaces écrasés, `\n` préservés) puis **détection par OCCURRENCE** des motifs d'injection (regex FR/EN + variantes paraphrasées, tolérant l'obfuscation inter-lettres `\s*`) ; b. neutralisation (`[[DONNEE-NON-EXECUTABLE]]`) — toute balise `<<<...>>>` déjà présente dans le document est neutralisée (motif `delimiteur_injecte`, ex. `<<<FIN-DONNEES>>>`, pour ne pas permettre la fermeture du bloc) ; c. balisage en bloc **donnée** : `<<<DONNEES-{jeton}>>> ... <<<FIN-{jeton}>>>` avec un **jeton aléatoire unique par session** (`secrets.token_hex(5)`, cf. `_generer_delimiteurs`) — impossible à anticiper par un document piégé. 3. si détections → événement `injection_detectee` dans le journal (**une entrée par occurrence** : les 14 instructions de `casB_injecte.md` → 14 entrées). 4. le `contexte_metier` (si fourni) subit le même traitement (G1) avant injection.

Phase 2 — boucle agent par agent (ordre `AG1→AG5`) :

Pour chaque agent `A` : 5. `_preparer_rag(A)` : les requêtes RAG de `REQUETES_RAG[A]` sont exécutées sur la base de connaissances ; les extraits (dédoublonnés, ≤ 6) sont injectés dans le prompt. 6. si `A == AG3` : `_preparer_cve_ag3(description_systeme)` complète le contexte — les mots-clés sont **extraits du document étudié** (`extraire_technologies`, cf. § 10) ; repli documenté si le document n'en contient aucun. 7. `agent.definir_contexte(contexte)` : l'agent reçoit les sorties normalisées des étapes précédentes (actifs, modèle, menaces, évaluations). 8. **Le prompt est assemblé** (`base.Agent.construire_prompt`) : `marqueur + consigne + outils + connaissances(RAG) + DONNEES + format attendu + rappels de sécurité` ; en cas de reprise, le bloc `=== ERREURS DE LA TENTATIVE PRÉCÉDENTE (à corriger impérativement) ===` est ajouté (la reprise n'est jamais un simple doublon). 9. `agent.produire()` → un appel LLM (`llm.complete(système, utilisateur)`) ; le prompt construit est mémorisé dans `agent._dernier_prompt` — sa longueur est journalisée comme **octets_entree réels** du journal. 10. le brut est parsé en JSON (`json.loads`), sinon `echec_json` → reprise (voir §8). 11. `toutes_les_erreurs(...)` : pile de validation (schéma, sources, `valide_par`, règles métier propres à l'agent). 12. si `A == AG4` et pas d'erreur : `_appliquer_niveaux_deterministes` recalcule chaque `niveau` via la matrice — **le LLM ne décide jamais du niveau** (G7). 13. si aucune erreur → `statut ok`, la sortie est mémorisée dans le workspace (`set_etape`) et **normalisée** pour l'étape suivante (`_injecter_etape`). 14. sinon → `echec_schema`, les erreurs sont réinjectées comme `erreurs_reparation` dans le prompt du prochain essai (retour de correction), jusqu'à 3 tentatives. 15. au-delà de 3 tentatives → **EtapeImpossibleError** : arrêt explicite (G8), aucun fichier de résultat n'est publié, le CLI sort en code 1.

Phase 3 — validation humaine (`valider_humainement`) : 16. `registre_provisoire()` : copie profonde des risques d'AG5 (le produit d'AG5 n'est **jamais muté**), avec `valide_par = null`. 17. pour chaque risque : question interactive `a/d/c` ; `c` (`_corriger_risque`) permet de modifier **actif, probabilité, impact, traitement, sources** et le texte de la menace — une correction de P/I **recalcule le niveau par la matrice** (jamais corrigé à la main, G7). 18. chaque décision → événement `validation_humaine` (accepte / diffère / corrige) ; chaque champ modifié → événement `correction_humaine` (champs + `valide_par`). Un risque **différé** garde `valide_par = null`.

Phase 4 — persistance : 19. `workspace.set_final(registre)` puis `workspace.sauvegarder()` → `workspace.json`. 20. `registre_final.json` est écrit ; événement `session_fin` horodaté.

6. Garde-fous de sécurité (G1→G10)

Correspondance exacte avec le référentiel du projet (p. 20 du sujet) et OWASP LLM :

#	Garde-fou	Mécanisme réel	Code	OWASP
G1	Filtrage des entrées (injection)	normalisation puis détection par occurrence (11 motifs FR/EN + paraphrases, obfuscation tolérée) + neutralisation + séparation consigne/donnée (délimiteurs aléatoires uniques par session) + <code>contexte_metier</code> assaini	<code>core/sanitizer.py</code> , agents 2-4)	LLM01 <code>agents/orchestrator.py</code> (étapes 1-4)
G2	Outils lecture seule	registre d'outils + <code>SandboxOutils</code> ; liste blanche par agent déléguée par l'orchestrateur (<code>_OUTILS_PAR_AGENT</code> , les agents n'appellent rien eux-mêmes, <code>PermissionError</code> sinon) ; <code>lire_fichier</code> restreint à <code>cases/</code>	<code>tools/registry.py</code> , <code>tools/file_reader.py</code> , agents 1-4	LLM06
G3	Anonymisation	cas d'étude factif ; avertissement explicite pour le mode réel ; zéro donnée réelle dans le pipeline	<code>README.md</code> , prompts	LLM02
G4	Sources citées	<code>sources</code> requis non vide par le schéma (enveloppe) et par risque AG5, avec contrôle de format : « aucune/néant/none/n/a/inconnu » rejetés, 3 à 120 caractères ; RAG renvoie <code>reference</code> à recopier	<code>core/validation.py</code> , <code>agents/knowledge.py</code>	LLM07 (anti-poisoning)

#	Garde-fou	Mécanisme réel	Code	OWASP
G5	Journalisation	journal JSON-lines horodaté de chaque appel, chaque outil, chaque décision	core/journal.py	—
G6	Validation humaine	valide_par écrit uniquement par validation humaine impose "type": "null" ; la chaîne "null" est rejetée	core/validation.py, core/schemas.py	agents/orch
G7	Niveau déterministe	matrice P×I appliquée par l'orchestrateur (AG4) et re-vérifiée sur chaque risque final (AG5) ; aucune « auto-autorisation » d'échelle inventée	tools/matrice.py, core/validation.py	agents/orch (overreliance)
G8	Arrêt bavard	3 tentatives max par étape, puis exception et code 1 — jamais de remplissage	agents/orchestrator.py, cli/main.py	LLM08 (excessive agency)
G9	Modèle remplaçable	interface FournisseurLLM (Protocol) ; dummy ■ openai_compat sans toucher au reste	llm/interface.py	—
G10	Connaissances versionnées, lecture seule	knowledge/*.md chargés en lecture seule et indexés ; les extraits sont des citations	tools/knowledge.py	LLM07

Règles métier complémentaires dans `validation.py` : - traitements limités à **réduire / transférer / éviter / accepter** — « ignorer », « rien », « néant » sont **interdits** ; - identifiants uniques (A-nn, M-nn, R-nn) ; - P/I **dans les échelles de configuration** (l'écho de l'agent est ignoré) ; - AG5 doit **recopier** P/I/niveau de l'étape 4 (jamais recalculer) et y rester rattaché par `menace_id`.

6.1 Les 11 motifs du sanitizer (G1)

`core/sanitizer.py` — `_MOTIFS_INJECTION` : paires (nom, regex) en français et en anglais, y compris des **variantes paraphrasées** tolérant l'obfuscation inter-lettres (`\s*`). La détection ne bloque pas l'analyse : chaque **occurrence** est **signalée** (`injection_detectee`) puis **neutralisée** en `[[DONNEE-NON-EXECUTABLE]]`. La détection et la neutralisation travaillent sur un texte **normalisé** (homoglyphes cyrilliques/grecs/fullwidth → latin, caractères zero-width et diacritiques combinants retirés, espaces écrasés, `\n` préservés) et se reportent aux indices **bruts** pour l'extrait journalisé.

Motif	Cible (exemple de ce qui est attrapé)
ignore_instructions	« ignore tes instructions », « disregard your previous rules »
disable_securite	« désactive tes consignes », « forget all security »
override_system	« tu es maintenant... », « you are now... », « new system prompt »
nouvelle_persona	« à partir de maintenant », « fais comme si », « pretend »
delimiteur_injecte	toute balise <<<...>>> insérée dans le document (ex. <<<FIN-DONNEES>>>, tentatives d'évasion du bloc de données)
delimiter_escape	<<SYS>>, <\ im_start\ >, <\ system\ >
data_as_instruction	une ligne instructions : ... en début de texte
jailbreak_mot	« jailbreak », « désactive tes règles »
assistant_prefix	une ligne assistant : ... insérée (usurpation de rôle)
disclose_secrets	« révèle tes instructions/clefs », « reveal your system prompt »
replay_prompt	« répète tout ce qui précède », « repeat everything above »

La détection est **par occurrence** : chaque instruction piégée produit une entrée. Vérifié par tests : les **14 instructions présentes dans casB_injecte.md** produisent **exactement 14 détections** (test_cas_injecte_exactement_14_detections, test_orchestracteur), dont l'obfusquée « ignore tes instructions » et la balise <<<FIN-DONNEES>>> (delimiteur_injecte) ; le document sain du cas B ne produit **aucun faux positif** (fix « désormais », « Dorénavant », « Consignes : » — test_document_legitime_sans_fausse_alarme).

7. La matrice déterministe (G7)

Matrice probabilité × impact **celle du sujet (p. 8)** :

Prob \ Impact	faible	moyen	élevé
élevée	moyen	élevé	critique
moyenne	faible	moyen	élevé
faible	faible	faible	moyen

Deux endroits en imposent le respect : 1. **AG4** : après validation, l'orchestracteur écrase `niveau` par `calculer_niveau(P, I)` — un niveau « inventé » est remplacé, jamais propagé. Une P/I hors matrice provoque un échec bloquant (pas de `continue` silencieux). 2. **AG5** : `verifier_niveaux_agent5` re-vérifie **chaque entrée du registre final** ; un niveau ≠ matrice est une erreur de validation (exception après 3 reprises).

8. Reprises et arrêt bavard (G8)

Boucle de reprise (`_executer_avec_reprises`) :

```

pour tentative = 1..3 :
1. appel LLM
    ■■ exception réseau/API → echec_llm → rejouer (même prompt)
    ■■ JSON invalide → echec_json → rejouer avec erreurs_reparation
    ■■ JSON valide → toutes_les_erreurs(...)
    ■■ 0 erreur → le niveau AG4 est recalé, retour sortie ■
    ■■ erreurs → echec_schema → rejouer avec erreurs_reparation
après 3 tentatives → EtapeImpossibleError (code 1, aucun livrable publié)

```

La **réparation** consiste à réinjecter les erreurs de la tentative précédente dans le prompt du tour suivant, sous le bloc `=== ERREURS DE LA TENTATIVE PRÉCÉDENTE (à corriger impérativement) ===` : l'agent sait *pourquoi* sa sortie a été refusée. Le dernier prompt construit est mémorisé sur `Agent._dernier_prompt` (c'est sa taille qui est journalisée en `octets_entree`). Les messages d'erreur sont **tronqués à 160 caractères** avant journalisation (pas de fuite de contenu LLM).

9. Recherche de connaissances (RAG) et base de connaissances (G10)

RAG minimaliste, sans dépendance vectorielle :

1. `knowledge/` contient 5 documents Markdown : `stride.md`, `linddun.md`, `iso27002.md`, `anssi.md`, `attck.md`.
2. À chaque démarrage, `BaseConnaissances` les charge et les **découpe en blocs** (par titre `#`).
3. `chercher_connaissance(requête)` : mots-clés (≥ 3 lettres, mots vides exclus) → score de cooccurrence par bloc → les **top extraits** sont renvoyés avec leur **référence** (`knowledge/stride.md` · `STRIDE` — grille d'identification).
4. L'orchestrateur pré-extraie **par agent** (requêtes fixes de `REQUETES_RAG`) et injecte ≤ 6 extraits dans le prompt.
5. L'agent est invité à **recopier ces références dans sources** — d'où la vérification G4.

C'est une forme de *grounding* : le modèle répond à partir d'un corpus contrôlé et citable, au lieu de puiser dans sa seule mémoire.

10. Le snapshot CVE

`data/cve_snapshot.json` : 5 CVE réelles avec score CVSS et référence NVD.

- `rechercher_cve(mots=..., version=...)` filtre par mots-clés ; **sans critère**, réponse vide.
- Utilisé pour **AG3** : `_preparer_cve_ag3(document)` **extrait les mots-clés technologiques du document étudié** via `extraire_technologies(document)` (`webrtc`, `tls`, `api`, `visio`, ... — l'extraction se fait sur la donnée, pas sur une liste codée en dur) ; si le document n'en contient aucun, un **repli documenté** est utilisé (`webrtc tls api`) et l'absence de mot-clé est assumée. (tests : `test_extraire_technologies_depuis_document`, `test_preparer_cve_repli_decentralise`).
- ■■ C'est un **instantané de démonstration**, figé : la fonction renvoie en toutes lettres qu'il ne faut pas en inventer d'autres et qu'il faut consulter le **NVD** en production. Cette limitation est volontaire et documentée (anti-hallucination).

11. Comparaison avec la référence manuelle (`--comparer`)

`src/core/comparaison.py` — `comparer_registres(genere, reference)` aligne deux registres par identifiant R-nn (ne mute aucun argument) et retourne un dict : `nb_reference`, `retrouves`, `inventes`, `oublies`, `ecarts_par_risque` (sur les champs `actif`, `probabilite`, `impact`, `niveau`, `traitement`), `nb_ecarts_niveau`, `taux_reconciliation` et `synthese` (prête à afficher).

- **Référence** : `data/reference_manuelle.json` (structure `{"risques":[...]}`, alimentée par `03_Analyse_Manuelle_Reference.md` — vérité terrain humaine).
- **CLI** : `python run.py run --case cases/casB_mediconsult.md --no-human --comparer` affiche la synthèse et les écarts détaillés ; `data/reference_manuelle.json` absent → le CLI le signale sans bloquer.
- **■ ■ ■ Honnêteté (tautologie assumée)** : en dry-run, `FournisseurSimule(fidele=True)` REPRODUIT la référence pour tester la chaîne — le score affiché ne prouve rien sur la qualité de l'analyse. Le CLI l'affiche explicitement : « mode dummy : comparaison tautologique par conception — relancer avec `--provider openai` pour une vraie comparaison agents ↔ humains ».
- **Preuve de non-tautologie** : `FournisseurSimule(fidele=False)` (tests) produit un registre **valide pour le pipeline** mais **divergent** — R-07 oublié, R-11 inventé, R-09 avec écart de niveau, R-08 avec traitement différent. `test_comparaison.py` vérifie que `comparer_registres` détecte précisément ces écarts (`oublies == ["R-07"]`, `inventes == ["R-11"]`, `nb_ecarts_niveau ≥ 1`).
- **Mode réel** : `--provider openai` lit `OPENAI_BASE_URL` (défaut `https://api.openai.com/v1`), `OPENAI_API_KEY` et `OPENAI_MODEL` (variables d'environnement uniquement) ; `temperature=0.2` par défaut (`src/llm/openai_compat.py`). Seule une exécution réelle permet une comparaison agents ↔ humains qui ait un sens.

12. Journalisation (G5)

`core/journal.py` — écriture **append-only** en JSON-lines, horodatage UTC ISO-8601.

Événements émis :

Événement	champs	émis par
<code>session_debut</code> / <code>session_fin</code>	<code>chemin</code> / <code>statut</code> , <code>nb_risques</code>	CLI, orchestrateur
<code>session_fin_motif</code>	<code>motif</code> (arrêt bavard)	CLI (erreur G8)
<code>appel_agent</code>	<code>agent</code> , <code>tentative</code> , <code>statut</code> (<code>ok</code> / <code>echec_schema</code> / <code>echec_json</code> / <code>echec_llm</code> / <code>outil</code>), <code>outils</code> , <code>octets_entree</code> / <code>octets_sortie</code> <code>réels</code> , <code>erreurs</code>	orchestrateur, sandbox
<code>injection_detectee</code>	liste { <code>motif</code> , <code>extrait</code> } — une entrée par occurrence	orchestrateur (G1)
<code>rag_echec</code>	<code>agent</code> , <code>erreur</code>	orchestrateur
<code>validation_humaine</code>	<code>risque</code> , <code>valide_par</code> , <code>décision</code> (<code>accepte</code> / <code>differe</code> / <code>corrige</code>)	orchestrateur
<code>correction_humaine</code>	<code>risque</code> , <code>valide_par</code> , champs modifiés (<code>menace</code> , <code>actif</code> , <code>probabilité</code> , <code>impact</code> , <code>traitement</code> , <code>sources</code>)	orchestrateur (S <code>_corriger_risque</code>)

13. Simulateur déterministe vs fournisseur réel

FournisseurSimule (dummy.py) - Sert les sorties figées qui correspondent à l'analyse manuelle de référence (14 actifs, STRIDE+LINDDUN, 10 menaces, 10 évaluations, 10 risques) — **pour tester la chaîne** : son `nom_produit` est explicite, `dummy-deterministic` (test de chaîne – n'est PAS une analyse). - Routage par **marqueur** dans le prompt (`__AGENT_1_INVENTAIRE__` ...). - Paramètre `fidele=True/False` : `False` produit un registre **valide pour le pipeline mais divergent** (R-07 oublié, R-11 inventé, R-09 écart de niveau, R-08 autre traitement) — utilisé par `test_comparaison.py` pour prouver que la comparaison n'est pas tautologique (cf. § 11). - Un mode `troubler=True` (tests uniquement) produit des sorties **volontairement invalides** (`valide_par` rempli, sources vides, traitement « ignorer », niveau faux) pour prouver que les garde-fous bloquent. - Usage : démo sans clé, tests déterministes. En dry-run, un affichage « 10/10 vs référence » ne prouve que la cohérence du pipeline — jamais la qualité de l'analyse.

OpenAICompatProvider (openai_compat.py) - `POST {base_url}/chat/completions` avec `messages=[system, user]`, `temperature=0.2` (défaut), `timeout=120`. - Clé et modèle **uniquement** via variables d'environnement (`OPENAI_BASE_URL`, `OPENAI_API_KEY`, `OPENAI_MODEL` — aucun secret en dur). - `extraire_json` (interface.py) : si la réponse est bavarde, extrait le premier `{...}` au dernier `}` ; sinon `ValueError` → reprise. - ■■ Ne jamais envoyer de **données réelles** (G3). Seul ce fournisseur rend la comparaison `--comparer` signifiante.

Remplaçabilité (G9) : tout objet implémentant `complete(système, utilisateur) → str` (contract `FournisseurLLM`) peut être branché sans modifier le pipeline.

14. Modèle de données

Dataclasses (core/models.py) — miroir exact des schémas JSON :

- **Actif** : `id`, `nom`, `type`, `description`, `criticite`, `valeur`, `proprietaire`, `contraintes`
- **Menace** : `id_menace`, `actif`, `frontiere`, `description`, `categorie`, `vulnerabilite`, `source_menace`
- **Evaluation** : `id_menace`, `probabilite`, `impact`, `niveau`, `justification`
- **Risque** : `id`, `actif`, `menace_id`, `menace`, `categorie`, `probabilite`, `impact`, `niveau`, `traitement`, `mesures`, `justification`, `sources`, `risque_residuel`, `proprietaire`, `valide_par`

L'**enveloppe** (`agent`, `resume`, `sortie`, `sources`, `incertitudes`) est le seul format qu'un agent peut rendre ; le workspace stocke ces enveloppes **validées et normalisées**.

15. Validation par agent : tableau de référence

Récapitulatif des vérifications appliquées à chaque étape (`toutes_les_erreurs`) :

Étape	Schéma	Sources ≥ 1 (format valide)	valide_par = null	Unicité ids	Autre règle
AG1	■	■	–	<code>id actifs</code>	<code>type</code> , <code>criticite</code> , <code>valeur</code> c les énumérations (via schéma)
AG2	■	■	–	–	échelles normalisées sans doublons

Étape	Schéma	Sources ≥ 1 (format valide)	valide_par = null	Unicité ids	Autre règle
AG3	■	■	–	<code>id_menace</code>	<code>frontiere</code> ∈ 1..6, <code>categorie</code> ∈ STRIDE/LINDDUN (via schéma)
AG4	■	■	–	<code>id_menace</code>	P/I dans les échelles de configuration ; niveau recalculé (pas d'écho)
AG5	■	■	■	<code>id</code>	<code>menace_id</code> requis + connu ; traitement autorisé (« ignorer » interdit) ; P/I/niveau = étape 4 ; niveau = matrice

16. Limites et points d'attention

1. **Sanitizer à base de motifs** : il détecte les injections *connues* (11 motifs FR/EN + paraphrases, § 6.1). Il ne prétend pas à l'exhaustivité — la **séparation consigne/donnée** (délimiteurs uniques par session) et la **validation stricte des sorties** sont les couches réellement décisives (défense en profondeur, à l'image du cours).
2. **CVE figées** (`cve_snapshot.json`) : ce sont des données de démo ; en production il faut interroger le NVD. La fonction refuse d'ailleurs d'en inventer.
3. **MAX_PROMPT_OCTETS** : constante définie dans `config.py` mais **pas encore exploitée** (le prompt est borné indirectement par `MAX_DOCUMENT_OCTETS`).
4. **RAG simple** : recherche par mots-clés, pas de plongements vectoriels — c'est un choix pédagogique (aucune dépendance, reproductibilité totale).
5. **Échelles normalisées** : AG2 choisit ses échelles « pour la vitrine » mais la validation utilise les échelles de `config.py` — volontaire : les 3 valeurs du sujet sont la référence commune.
6. **Dry-run vs réel** : le simulateur reproduit la référence manuelle pour tester la chaîne — il **n'est pas** une analyse, et la comparaison `--comparer` en dry-run est **tautologique** (le CLI le rappelle). Seul le mode réel (`--provider openai`) fait varier les réponses : c'est la seule comparaison qui ait un sens, complétée par la grille C1→C10 du jalon 4. La non-tautologie de l'outil de comparaison est prouvée par le mode `fidele=False` (écarts détectés, cf. § 11).

17. Glossaire

Terme	Définition
Agent	composant IA qui produit une étape de l'analyse (un prompt + outils + contrat de sortie)
Orchestrateur	coordonne les agents : contexte, validation, reprises, mémoire, journal
Enveloppe	contrat JSON commun de sortie (agent , resume , sortie , sources , incertitudes)
RAG	<i>Retrieval-Augmented Generation</i> : injection d'extraits d'une base de connaissance dans le prompt
Sanitizer	filtre d'entrée : détecte/neutralise les tentatives d'injection de prompt
Matrice P×I	outil déterministe « probabilité × impact → niveau » (le LLM n'en décide pas)
Arrêt bavard	en cas de rejet répété (3×), l'outil s'arrête en erreur au lieu d'inventer
valide_par	champ signé par l'humain uniquement ; null tant que rien n'est validé
Sémantique des traitements	réduire / transférer / éviter / accepter ; « ignorer » toujours interdit
Snapshot CVE	jeux de données CVE/CVSS local et figé, à rafraîchir via le NVD en production
Comparaison (--comparer)	alignement du registre généré vs data/reference_manuelle.json ; tautologique en dry-run, signifiante en mode réel
Référence manuelle	data/reference_manuelle.json , vérité terrain humaine issue de 03_Analyse_Manuelle_Reference.md

Références croisées : README.md (vue raccourcie), GUIDE_UTILISATION.md (quoi faire), ../02_Architecture_MultiAgents.md (conception), ../03_Analyse_Manuelle_Reference.md (référence manuelle), ../04_Tests_Comparaison_Agents.md (résultats de tests), ../07_Guide_Repetition_Soutenance.md et ../08_Demo_Scriptee.md (préparation soutenance), ../utils/md_to_pdf.py (génération des PDF des livrables).